LatticeCache Python SDK

ar028 · 14 August 2026

LatticeCache is a fictional Python library for caching deterministic function calls on disk. It is intentionally small, typed, and boring: the same inputs produce the same cache key, and expired values disappear without ceremony.

Installation

LatticeCache requires Python 3.11 or later. Install it from the imaginary package index:

```
pip install latticecache
```

Quick start

Create one cache and call `get_or_set` with a stable key. The factory runs only when no live value exists.

```
from datetime import timedelta
from latticecache import Cache

cache = Cache(".cache/weather")

forecast = cache.get_or_set(
    "madrid:2026-08-14",
    factory=lambda: fetch_forecast("Madrid"),
    ttl=timedelta(minutes=15),
)
```

Values are encoded as JSON by default. Supply a codec when storing dataclasses, NumPy arrays, or other objects that JSON cannot represent.

Configuration

Option	Default	Meaning
`namespace`	`"default"`	Prefix used to isolate cache keys
`max_bytes`	`256 MiB`	Soft size limit checked after each write
`serializer`	`"json"`	Built-in `json`, `text`, or a custom codec
`read_only`	`False`	Allow reads but never create or refresh entries

Configuration may be passed to `Cache` directly or loaded from `pyproject.toml` under `[tool.latticecache]`. Constructor arguments win when both are present.

API reference

`Cache`

A filesystem-backed cache. Creating an instance makes its directory lazily on the first write; constructing a read-only cache never changes the filesystem.

Parameters

Name	Type	Description
`path`	`str | Path`	Directory containing cached values and metadata
`namespace`	`str`	Logical partition included in every derived key
`max_bytes`	`int`	Approximate limit before least-recently-used eviction
`read_only`	`bool`	Disable writes, refreshes, and eviction

`Cache.get_or_set`

Return the live value stored under `key`. If the key is absent or expired, call `factory`, store its result atomically, and return the new value.

- **Raises `ReadOnlyMiss`** when a read-only cache has no live value.
- **Raises `EncodeError`** when the configured serializer rejects the factory result.
- The factory's own exceptions pass through unchanged and are never cached.

`Cache.invalidate`

Remove one key. Returns `true` when an entry existed and `false` when there was nothing to do.

`cached`

Decorate a deterministic function. By default, the key combines the function's qualified name, its source fingerprint, and a canonical encoding of positional and keyword arguments.

```
from latticecache import Cache, cached

cache = Cache(".cache/models", namespace="v2")

@cached(cache, ttl=3600)
def load_model(model_id: str, *, quantized: bool = False):
    return download_model(model_id, quantized=quantized)
```

Mutable global state is not part of the derived key. Pass every input explicitly or provide a custom `key` function; invisible dependencies and caches are natural enemies.